

CS-3510-B Exam 4 (Set B)

Elijah Martin McCauley

TOTAL POINTS

79 / 110

QUESTION 1

Problem 1 30 pts

1.1 (i) 3 / 3

✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.2 (ii) 3 / 3

✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.3 (iii) 3 / 3

✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.4 (iv) 3 / 3

✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.5 (v) 3 / 3

✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.6 (vi) 3 / 3

✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.7 (vii) 3 / 3

✓ - 0 pts Correct (True)

- 3 pts Incorrect

1.8 (viii) 3 / 3

✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.9 (ix) 3 / 3

✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.10 (x) 0 / 3

- 0 pts Correct (False)
✓ - 3 pts Incorrect

QUESTION 2

2 Problem 2 17 / 25

- 0 pts Correct
- 17 pts Completely Incorrect/ Missing
- 17 pts Reverse reduction provided (k-Clique
\$\$\rightarrow\$\$ NP-complete problem)
- 12 pts Incorrect Reduction/ Missing
- 5 pts Incorrect/ Missing NP proof
- 3 pts Major Error in NP Proof
- 2 pts Minor Error in NP Proof
- 5 pts Minor Mistake in Reduction (ex. small
mistake in converting Vertex Cover to k-Clique)
✓ - 8 pts Major Error in Reduction (ex. reduction
cannot be justified)

- **2.5 pts** Incorrect/ Missing $x \in A \implies f(x)$

$x \in B$

- **2.5 pts** Incorrect/ Missing $x \notin A \implies f(x) \notin B$

QUESTION 3

3 Problem 3 10 / 10

✓ - **0 pts** Correct

- **10 pts** Completely Incorrect (Proved NP-C)/ Non poly-time algorithm provided/ Missing

- **3 pts** Minor Error

- **7 pts** Major Error (ex. More than 2 edge cases exist)

QUESTION 4

4 Problem 4 25 / 25

✓ - **0 pts** Correct

- **25 pts** Completely Incorrect/ Missing

- **17 pts** Reverse reduction provided (Set-Disjoint $\implies$ NP-complete problem)

- **12 pts** Incorrect Reduction/ Missing

- **5 pts** Incorrect/ Missing NP proof

- **3 pts** Major Error in NP Proof

- **2 pts** Minor Error in NP Proof

- **5 pts** Minor Mistake in Reduction (ex. small mistake in converting Independent Set to Set-Disjoint)

- **8 pts** Major Error in Reduction (ex. reduction cannot be justified)

- **2.5 pts** Incorrect/ Missing $x \in A \implies f(x) \in B$

- **2.5 pts** Incorrect/ Missing $x \notin A \implies f(x) \notin B$

QUESTION 5

5 Problem 5 0 / 10

- **0 pts** Correct

✓ - **10 pts** Completely Incorrect (Proved NP-C)/ Missing

- **3 pts** Minor Error

- **7 pts** Major Error (ex. Incorrect reduction to 2-SAT/ More than 1 edge cases exist/ Non poly-time algorithm provided)

1 this reduction attempts to show that 3-Unique-SAT must be harder than 2-SAT, but we want to show the opposite: that it is easier since 2-SAT is in P.

2 this reduction is incorrect

3 the correct reverse reduction: given a 2-SAT clause $(x_1 \vee x_3)$, convert it to $(x_1 \vee x_3 \vee x_2) \wedge (x_1 \vee x_3 \vee \neg x_2)$ which has the same truth value no matter the value of x_2 .

4 if the 2-SAT input is $(x_1 \vee \neg x_1)$ then this is unsatisfiable, but adding x_2 for $(x_1 \vee \neg x_1 \vee x_2)$ is satisfied if x_2 is true.

QUESTION 6

6 Problem 6 0 / 10

- **0 pts** Correct

✓ - **10 pts** Completely Incorrect/ Missing/ Reverse reduction provided (Experimental Cuisine $\implies$ NP-complete problem)

- **9.5 pts** Only Reduction provided from Clique, Vertex Cover, Subset-Sum or Knapsack

- **8.5 pts** Reduction from Clique, Vertex Cover,

Subset-Sum or Knapsack with complete bi-directional proof.

- **4 pts** Mistake in Reduction (ex. small mistake in converting Independent Set to Experimental Cuisine)

- **2.5 pts** Incorrect/ Missing $x \in A \implies f(x) \in B$

- **2.5 pts** Incorrect/ Missing $x \notin A \implies f(x) \notin B$

GTID: 903684653
NAME: Elijah McCauley

Georgia Institute of Technology

Spring 2023

CS 3510B – Design & Analysis of Algorithms
Exam 4 (Set B)

April 18, 2023

TIME ALLOWED: 75 MINS

Name: Elijah McCauley

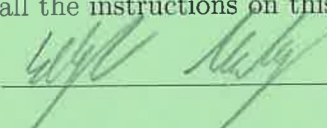
GTID: 903684653

GT Username: emccauley6

INSTRUCTIONS TO CANDIDATES

1. Please write your NAME and GTID clearly on all the pages.
2. This examination paper is worth **110 points** (100 + 10 extra credit points) comprising of **SIX (6)** questions and **TWELVE** printed pages.
3. **ONLY** write on the front sheets of paper that are numbered. The backs will not be scanned.
4. Wherever a blank is provided, please write your answer in the given blank.
5. Calculators are **NOT** allowed.

I am in aware of and the accordance with Academic Honor Code of Georgia Tech and the Georgia Tech Code of Conduct. I'll use no external help on this test. Also, I have read all the instructions on this page.

Signature: 

GTID: 403684653
NAME: Elijah McClary

You can only use the following NP-Complete problems for the reductions in this exam:

1. SAT
2. 3-SAT
3. Independent Set
4. Clique
5. Vertex Cover
6. Knapsack
7. Subset Sum
8. 3-Coloring

Note: Feel free to use the rest of this page to extend your answer to any questions or for rough work. In case you're using it for answer extension, please make sure to make a reference.

Problem 1 (30 points; 3 points each)

Indicate whether the following statements are *true* or *false*.

(i) Consider

$$f(x_1, x_2, x_3, x_4, y) = (\overset{F}{x_1} \vee \overset{F}{x_2} \vee y) \wedge (\overset{T}{\bar{y}} \vee \overset{F}{x_3} \vee x_4)$$

and

$$g(x_1, x_2, x_3, x_4, y, z) = (\overset{F}{x_1} \vee \overset{F}{x_2} \vee z) \wedge (\bar{z} \vee x_3 \vee y) \wedge (\bar{y} \vee \bar{x}_4)$$

f and g will have identical results for all assignments x_1, x_2, x_3 regardless of y and z .

Answer: F

(ii) If we find a polynomial time algorithm to solve Knapsack, this gives us a polynomial time algorithm for the Independent Set problem.

Answer: T

(iii) If we find a polynomial time algorithm to solve Independent Set, this gives us a polynomial time algorithm for the Knapsack problem.

Answer: T

(iv) Consider a variation of the Subset-Sum problem called Binary-Subset-Sum which takes in a target k and an array $A = [x_1, \dots, x_n]$ where $x_i \in \{0, 1\}$, and returns an output subset $S \subseteq A$ such that $\sum_{s \in S} s = k$.

Is Binary-Subset-Sum an NP-Complete problem?

Answer: F

(v) If there exists a reduction from problem A to problem B , then we prove that problem B is at least as hard as problem A to solve.

Answer: T

(vi) If $\text{Subset-Sum} \in P$ then $\text{Knapsack} \in P$.

Answer: T

✓ (vii) Suppose you choose any NP-Complete problem H . It is possible that you can verify a candidate solution S to H in polynomial time.

Answer: T

(viii) We have n problems $A_1, A_2, \dots, A_n$. If you can reduce $A_1 \rightarrow A_2, A_2 \rightarrow A_3, \dots$, and $A_{n-1} \rightarrow A_n$, then there must exist a reduction from $A_1 \rightarrow A_n$.

Answer: T

(ix) If we find a polynomial time algorithm to solve Independent Set, this gives us a polynomial time algorithm for BOTH the Vertex Cover and the Clique problem.

Answer: T

(x) Suppose problem L is in P . If you reduce L to another problem M using a polynomial-time transformation, M must be in P .

Answer: T

GTID: 903684653
NAME: Elijah McCarty

Problem 2 (25 points)

A

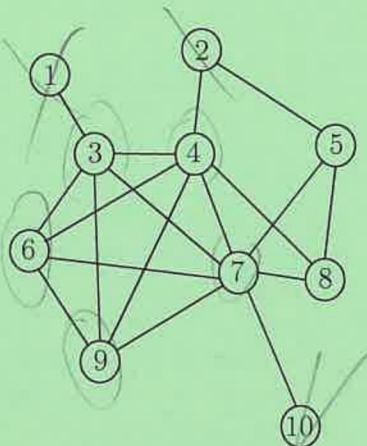
Show that the following k -Clique problem is NP-Complete by reducing it from the Vertex Cover problem.

We proved k -Clique and Vertex Cover are NP-Complete from Independent Set. This question is asking you to reprove that k -Clique is NP-complete by reducing from Vertex Cover.

Note: Reducing from any other NP-Complete problems will receive zero points.

Input: An undirected graph $G = (V, E)$ and a positive integer k .

Example Input: Let $G = (V, E)$ be an undirected graph below. $(G, k = 5)$ is the input.



Output: Return True if a subset $V' \subseteq V$ corresponding to the k -sized clique exists. If a clique of size at least k doesn't exist, return False.

Example Output: True since $V' = \{3, 4, 7, 9, 6\}$

(i) Show the problem is in class NP.

To show it is NP we need to be able to verify a solution in polytime. given a witness V' , we can simply check that it is length k if not return False. if $|V'| \geq k$ then we loop over the edges in the G , graph to confirm that there is an edge between each of the vertices in V' . this take $O(|E|)$ since we loop over edges.

VC points which connect to each edge

GTID: 903684653
NAME: Elijah McCauley

VC 3

(ii) Show the problem is in class NP-Hard.

To show it is NP hard, we need to reduce it from Vertex cover. Vertex cover takes in a graph

$G = (V, E)$ and k where k is the max vertices allowed to create the vertex cover. To reduce our input, we need to make a G' to input into k clique. We take ~~the vertices and~~ loop through the vertices and remove ones with only one edge ^{and remove that edge}. We then add edges between the remaining vertices. This means that the ^{remaining} vertices will form a clique of size k .

VC 3

If there is a valid vertex cover that can reach all edges with k points, then after our reduction, those k points will form our k clique.

If there is not a vertex cover, then ~~some~~ will not be able to form a large enough clique in our reduction.

if 5 points can

$$\left(\overset{F}{x_1} \vee \overset{T}{\bar{x}_2} \right) \wedge \left(\overset{F}{x_2} \vee \overset{T}{\bar{x}_1} \right)$$

GTID: 903684653
NAME: Elijah McCullum

Problem 3 (10 points)

$$\left(\overset{F}{x_1} \vee \overset{T}{\bar{x}_2} \right) \left(\overset{F}{x_2} \vee \overset{T}{\bar{x}_1} \right)$$

Show that the following Positive-SAT problem is in class P.

Input: A boolean formula in CNF with m clauses and n variables, but all the literals are non-negated i.e. they don't contain a variable of the form $\bar{x}_i$ for $i \leq n$. That is literal $\bar{x}_i$ doesn't exist in any of the m clauses.

Output: A satisfying assignment such that each of the m clauses evaluates to True.

To prove it is in class P we need to reduce from another problem in class P like 2 sat.

We start with our input into 2 sat which has m clauses of length 2. We need to loop through the m clauses input into 2 sat

and get rid of any negated literals without affecting the value of the literal. We can do this by replacing each negated literal with

a new variable y so $(x_1 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_1)$ becomes $(x_1 \vee y_2) \wedge (x_2 \vee y_1)$ where

y_1 will be at the negation of x_1 and so on.

This is a bit of a trivial solution, but it runs in poly time and if the input to 2 sat works, so does this reduction and

vice versa

$$\left(\overset{F}{x_1} \vee \overset{T}{\bar{x}_2} \right) \wedge \left(\overset{F}{x_2} \vee \overset{T}{\bar{x}_1} \right)$$

$$(x_2 \vee y_2)$$

GTID: 901684653
NAME: Elijah McCrory

Problem 4 (25 points)

Show that the following Set-Covering problem is NP-Complete.

Input: Subsets $S_1, S_2, \dots, S_m$ of a large set U , a positive integer $k \leq m$.

Output: If k subsets exists among $S_1, S_2, \dots, S_m$ such that each two are pairwise disjoint i.e. the intersection of any two sets from the k outputted sets is empty. output True. If no selection of k subsets exists, output False.

zu Khasno
overlapping

(i) Show the problem is in class NP.

To verify ^{a solution to the problem} ~~the problem~~ in poly time,
we take a witness which is k outputted
sets and loop over the values ^{each} $\text{in } k$ and compare them
to the values in each other returned subset.

This would take $O(k \cdot k \cdot |k|)$ which is inefficient but still poly time. No value is in any of the k subsets can be repeated, so we need compare each value returned to every other one.

(ii) Show the problem is in class NP-Hard.

To show this problem is NP-Hard we must reduce from a known NP-Complete problem, for this let's use Indset. Indset takes in a graph $G(V, E)$ and a minimum value k . We can take this graph and say that each ~~node~~ vertex is a subset ^{of length 2}, and if there is an edge between 2, they have the same value in their subset. This reduction takes $O(1)$ because we are not altering the input graph, simply defining the meaning of vertices and edges.

If there is an indset, it means there are k vertices that don't share an edge, because the vertices are ~~subgroups~~ subsets, and edges mean their values are the same, this means there ^{are} k subsets that exist and do not share any values with each other.

If there is not an indset, too many ~~subgroups~~ ^{subsets} share values, some cannot make k subsets that have no intersection because there are not k vertices that don't share an edge.

Problem 5 (10 points)

Show that the following New-Three-SAT problem is in class P.

Input: A boolean formula in CNF with m clauses and n variables, but each clause C_i has exactly three literals and each clause contains at least one of these two literals: x_2 or $\bar{x}_2$.

Example Input: $(x_2 \vee x_4 \vee x_2) \wedge (x_3 \vee x_1 \vee \bar{x}_2) \wedge (x_3 \vee x_5 \vee \bar{x}_2)$. Note that each clause in the input has variable x_2 .

Output: A satisfying assignment such that each of the m clauses evaluates to True.

To show it is in class P we must reduce from a known P problem, 2 sat. We take our input in 2 sat which has 2 literals per clause, so the form $(x_1 \vee x_2) \wedge (x_1 \vee \bar{x}_3)$ for example. To convert this to New 3 sat, we simply need to loop over the M clauses in the input, and add an x_2 to it, so $(x_1 \vee x_2) \wedge (x_2 \vee \bar{x}_3) \Rightarrow (x_1 \vee x_2) \wedge (x_1 \vee \bar{x}_3 \vee x_2)$ which we will always set to true.

reassign any x_2 to be called y

This way, if there was a successful assignment of variables before the reduction, that same assignment works for the reduction except that ^{original} x_2 is now y and x_2 is set to true. if there was not a valid assignment, there still isn't because adding the x_2 did not change anything.

running in $O(m)$

Problem 6 (Extra Credit; 10 points)

Show that the following Experimental Cuisine problem is NP-Hard.

We are feeling experimental and want to create a new dish. There are various ingredients we can choose from and we'd like to use as many of them as possible, but some ingredients don't go well with others. If there are n possible ingredients (numbered 1 to n), we write down an $n \times n$ matrix giving the discord between any pair of ingredients. This discord is a real number between 0.0 and 1.0, where 0.0 means "they go together perfectly" and 1.0 means "they really don't go together." Here's an example matrix when there are five possible ingredients.

	1	2	3	4	5
1	0.0	0.4	0.2	0.9	1.0
2	0.4	0.0	0.1	1.0	0.2
3	0.2	0.1	0.0	0.8	0.5
4	0.9	1.0	0.8	0.0	0.2
5	1.0	0.2	0.5	0.2	0.0

In this case, ingredients 2 and 3 go together pretty well whereas 1 and 5 clash badly. Notice that this matrix is necessarily symmetric; and that the diagonal entries are always 0.0. Any set of ingredients incurs a penalty which is the sum of all discord values between pairs of ingredients. For instance, the set of ingredients $\{1, 3, 5\}$ incurs a penalty of $0.2 + 1.0 + 0.5 = 1.7$. We want this penalty to be small.

Input: D, n, p ; where n is the number of ingredients to choose from, D is the $n \times n$ "discord" matrix and p is some number such that $p \geq 0$.

Output: The maximum number of ingredients we can choose with penalty $\leq p$.

it is NP because we can verify a solution in poly time by taking a witness of ingredients and adding up their penalties to ~~check~~ check if it's greater than p if so take it true,

GTID: _____
NAME: _____